Coding Interview Cheat Book (Python)

Consolidated from two files: `leetcode_concise_solutions.md` (LeetCode set) and `online_question.py` (Confluent and IBM interview reports, notes, and code). Each question has the same layout so it is easy to reproduce under time pressure: **Question, Example, Approach (numbered steps), Python, Complexity (time and space)**.

Problem statements are short paraphrases. Entries marked **(reconstructed)** were vague in the source, so state your assumptions and ask the interviewer.

How to use this in an interview

Work in this order: **Clarify -> Plan -> Execute -> Test** (the routine from your notes).

1. **Clarify:** inputs and sizes, duplicates, ordering of timestamps, boundary rules (`<` or `<=`), empty and unknown cases, "is O(1) average OK?".
2. **Plan:** say the data structure and the complexity out loud before typing.
3. **Execute:** write the simplest correct version first, then optimize if asked.
4. **Test:** run the given example, then an empty input, one element, duplicates, and a boundary value.

Pattern cheat sheet (signal -> tool)

If you see...	Reach for...	Examples
"last N", "within the last K seconds"	deque, OrderedDict, two pointers	Q9, Q10, Q15, Q11
need min / max / k-th while data changes	heap (lazy deletion if values get overwritten)	Q16, Q19, Q21, Q22, Q6, Q23
lookup by key + ordering by time or recency	hash map + linked list / OrderedDict	Q1, Q2, Q3
"earliest time that fits", sorted arrays	sort + two pointers or binary search	Q18, Q5, Q11
answer is monotonic (more strength / time never hurts)	binary search on the answer	Q46, Q47
all combinations / grids / puzzles	backtracking with pruning	Q34, Q35, Q36, Q31, Q33
count ways, min cost, can it be built	DP (1D / 2D / bitmask / memo)	Q37, Q41, Q40, Q39, Q38, Q28
dependencies, shortest path	topological sort (Kahn), Dijkstra	Q42, Q43, Q44
search text, prefixes, wildcards	inverted index, trie	Q24, Q25, Q29, Q30

(Q numbers match the question headings below.)

Python toolbox: what things cost

Operation	Cost
`dict` / `set` get, set, in	O(1) average
`list.append()`, `list.pop()`	O(1)
`list.pop(0)`, `list.insert(0, x)`	O(n), use `deque` instead
`bisect_left` / `bisect_right`	O(log n)
`bisect.insort`	**O(n)** (the search is O(log n) but shifting is O(n))
`heapq.heappush` / `heappop`	O(log n)
`heapq.heapify`	O(n)
`deque.append` / `popleft`	O(1)
`OrderedDict.move_to_end` / `popitem(last=False)`	O(1)
`sorted()`	O(n log n)
slicing `s[i:j]`	O(j - i)

`heapq` is a **min**-heap: store `-value` for a max-heap. Put a tie-breaker index in heap tuples when the payload cannot be compared.

Question index

#	Question	Source	Pattern	Time	Space
Q1	LRU Cache (LeetCode 146, Medium)	LeetCode 146	Hash map + doubly linked list	O(1) per op	O(capacity)
Q2	LFU Cache (LeetCode 460, Hard)	LeetCode 460	Hash maps + OrderedDict per frequency	O(1) per op	O(capacity)
Q3	Insert Delete GetRandom O(1) (LeetCode 380, Medium)	LeetCode 380	Hash map + array	O(1) average per op	O(n)
Q4	Random Queue (+ equality, thread safety, RLE equality)	Confluent	Array with swap-with-last	enqueue/dequeue O(1) expected, equality O(n), RLE equality O(R+V)	O(n)
Q5	Time Based Key-Value Store (LeetCode 981, Medium)	LeetCode 981	Hash map + binary search	set O(1); get O(log n)	O(total entries)
Q6	Key-Value Store with Get, GetAverage and GetMax	Confluent	Dict + running sum + lazy-deletion max-heap	put O(log n), get O(1), average O(1), max amortized O(log n)	O(n + updates)
Q7	Design Authentication Manager (LeetCode 1797, Medium)	LeetCode 1797	Hash map with expiry	generate / renew O(1); count O(n)	O(n)
Q8	Token Manager: generate / renew / list active tokens by create_time	Confluent ("Retrieve Token List")	Dict + sort on read	generate/renew O(1), list O(n log n)	O(n)

#	Question	Source	Pattern	Time	Space
Q9	Sliding-Window Cache: put / get / average	Confluent qualifier (senior)	OrderedDict + running sum	O(1) amortized per op	O(n)
Q10	Message Logger (10-Second Rate Limit per Message)	Confluent	Hash map of last printed time	O(1) per call	O(M) distinct messages
Q11	Silent Sensor Detector (STABLE / UNSTABLE)	Confluent	Per-sensor sorted list + binary search	record O(P) worst (O(1) if in order), query O(log P)	O(P)
Q12	Cache With Time Limit (LeetCode 2622, Medium; JS-only on LeetCode, Python port)	LeetCode 2622	Hash map with expiry	set / get O(1); count O(n) with lazy expiry	O(n)
Q13	Debounce (LeetCode 2627, Medium; JS-only on LeetCode, Python port)	LeetCode 2627	Timer	O(1) per call	O(1)
Q14	Promise Time Limit (LeetCode 2637, Medium; JS-only on LeetCode, Python port)	LeetCode 2637	Async timeout	O(1) overhead per call	O(1)
Q15	Count Events in the Window (t − W, t] for Each Event	Confluent / PracHub	Two pointers on a sorted array	O(n)	O(1) extra
Q16	Meeting Rooms II (LeetCode 253, Medium)	LeetCode 253	Sort + min-heap	O(n log n)	O(n)

#	Question	Source	Pattern	Time	Space
Q17	Meeting Rooms III (LeetCode 2402, Hard)	LeetCode 2402	Two heaps	O(m log m + m log n)	O(n + m)
Q18	Meeting Scheduler (LeetCode 1229, Medium)	LeetCode 1229	Sort + two pointers	O(n log n + m log m)	O(1) extra
Q19	Kth Largest Element in an Array (LeetCode 215, Medium)	LeetCode 215	Min-heap of size k	O(n log k)	O(k)
Q20	Top K Frequent Elements (LeetCode 347, Medium)	LeetCode 347	Counter + bucket sort	O(n)	O(n)
Q21	Find Median from Data Stream (LeetCode 295, Hard)	LeetCode 295	Two heaps	addNum O(log n); findMedian O(1)	O(n)
Q22	Merge k Sorted Lists (LeetCode 23, Hard)	LeetCode 23	Min-heap	O(N log k), N = total nodes	O(k)
Q23	Minimum Health for Gaming (rank-th strongest each level)	Confluent (reconstructed)	Min-heap of size `rank`	O((n + L) log rank)	O(rank)
Q24	Search a Word or Phrase in Many Documents (+ AND / OR / brackets)	Confluent onsite	Inverted index with positions	build O(T), query about O(postings of the query words)	O(T)
Q25	Text Search Engine: WORD	Confluent (PracHub)	Counter + sliding	WORD O(1), PHRASE O(T·P) per query	O(T)

#	Question	Source	Pattern	Time	Space
	and PHRASE Counts		window		
Q26	Unix `tail -N` (Last N Lines of a Huge File)	Confluent onsite	Read blocks backwards / bounded deque	O(size of the last N lines), not O(file)	O(N lines + one block)
Q27	Function Overload Matching (with Variadic Parameters)	Confluent onsite	Rule-based scan	O(N·M) per lookup	O(N·M)
Q28	Wildcard Matching (LC 44) and Regular Expression Matching (LC 10)	Glassdoor (Confluent reports "pattern matching")	2D DP	O(m·n)	O(m·n)
Q29	Design Add and Search Words Data Structure (LeetCode 211, Medium)	LeetCode 211	Trie	add O(L); search O(L), worst O(26^L) with dots	O(total characters)
Q30	Word Search II (LeetCode 212, Hard)	LeetCode 212	Trie + DFS	O(R·C·3^L) worst case, far less with pruning	O(total word characters)
Q31	Word Search (LeetCode 79, Medium)	LeetCode 79	DFS backtracking	O(R·C·3^L)	O(L) recursion
Q32	Valid Sudoku (LeetCode 36, Medium)	LeetCode 36	Hash set	O(81) = O(1)	O(81) = O(1)
Q33	Sudoku Solver (LeetCode 37, Hard)	LeetCode 37	Backtracking	exponential worst case, fast in practice with pruning	O(81)

#	Question	Source	Pattern	Time	Space
Q34	Combination Sum (LeetCode 39, Medium)	LeetCode 39	Backtracking	exponential, bounded by the number of combinations	O(target / min) recursion depth
Q35	Combination Sum II (LeetCode 40, Medium)	LeetCode 40	Backtracking + skip duplicates	O(2^n) worst case	O(n)
Q36	Combination Sum III (LeetCode 216, Medium)	LeetCode 216	Backtracking	O(C(9,k)·k)	O(k)
Q37	Combination Sum IV (LeetCode 377, Medium)	LeetCode 377	1D DP (ordered sequences)	O(target·len(nums))	O(target)
Q38	Warehouse Loading / Bank Transactions: Can a Prefix or Subset Reach a Target?	Confluent / IBM	Subset-sum with a set of reachable sums	O(N·S)	O(S)
Q39	Best Price with Value Meals (up to 3 unique wanted items)	Confluent (LeetCode discuss)	Bitmask DP	O(2^k · offers), $k \leqslant 3$	O(2^k)
Q40	Minimum Number of Days to Eat N Oranges (LeetCode 1553, Hard)	LeetCode 1553	Memoized recursion	$O(\log^2 n)$	$O(\log^2 n)$
Q41	Word Break (LeetCode 139, Medium)	LeetCode 139	1D DP	O(n·L), L = longest word	O(n)
Q42	Course Schedule (LeetCode 207,	LeetCode 207	Graph, topological	O(V + E)	O(V + E)

#	Question	Source	Pattern	Time	Space
	Medium)		sort (Kahn)		
Q43	Course Schedule II (LeetCode 210, Medium)	LeetCode 210	Graph, topological sort (Kahn)	O(V + E)	O(V + E)
Q44	Minimum Monster Cost in a Grid (+ path)	Confluent VO (reconstructed)	Dijkstra	O(R·C·log(R·C))	O(R·C)
Q45	Eliminate Maximum Number of Monsters (LeetCode 1921, Medium)	LeetCode 1921	Greedy + sort	O(n log n)	O(n)
Q46	Minimum Initial Strength to Defeat All Monsters (LeetCode 4008, Medium)	LeetCode 4008	Binary search on answer + difference array	O(n + b + n log S), S = sum(monsters)	O(n)
Q47	Minimum Time to Eat All Grains (LeetCode 2604, Hard)	LeetCode 2604	Binary search on answer + greedy sweep	O(n log n + m log m + (n + m) log T)	O(1) extra
Q48	Minimum Value to Get Positive Step-by-Step Sum	LeetCode 1413	Prefix sum minimum	O(n)	O(1)

Part 1: Hash Maps, Design and Caching

Q1. LRU Cache (LeetCode 146, Medium)

Source: LeetCode 146 | Pattern: Hash map + doubly linked list

Question: Design a cache with capacity `capacity`. `get(key)` returns the value or `-1`. `put(key, value)` inserts or updates, and evicts the least recently used key when over capacity. Both must be O(1).

Example: capacity 2: `put(1,1)`, `put(2,2)`, `get(1)` is 1, `put(3,3)` evicts 2, `get(2)` is -1.

Approach (short): `OrderedDict` keeps order for you.

python

```python
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.cap, self.d = capacity, OrderedDict()

    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]

    def put(self, key, value):
        if key in self.d:
            self.d.move_to_end(key)
        self.d[key] = value
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

Approach (interview version): hash map + doubly linked list with dummy head and tail. Most recent goes next to the head, evict from before the tail.

python

```python
class Node:
    __slots__ = ("k", "v", "prev", "next")
    def __init__(self, k=0, v=0):
        self.k, self.v, self.prev, self.next = k, v, None, None

class LRUCache:
    def __init__(self, capacity):
        self.cap, self.map = capacity, {}
        self.head, self.tail = Node(), Node()
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, n):
        n.prev.next, n.next.prev = n.next, n.prev

    def _add_front(self, n):
        n.prev, n.next = self.head, self.head.next
        self.head.next.prev = n
        self.head.next = n

    def get(self, key):
        if key not in self.map:
            return -1
        n = self.map[key]
        self._remove(n); self._add_front(n)
        return n.v

    def put(self, key, value):
        if key in self.map:
            n = self.map[key]
            n.v = value
            self._remove(n); self._add_front(n)
            return
        if len(self.map) == self.cap:
            lru = self.tail.prev
            self._remove(lru)
            del self.map[lru.k]
        n = Node(key, value)
        self.map[key] = n
        self._add_front(n)
```

Complexity: Time: O(1) per op | Space: O(capacity)

Q2. LFU Cache (LeetCode 460, Hard)

Source: LeetCode 460 | Pattern: Hash maps + OrderedDict per frequency

Question: Design a cache with `get` and `put`, both O(1). When full, evict the **least frequently used** key. Break ties by evicting the **least recently used** among them. Every `get` or `put` on a key counts as a use.

Example: capacity 2: `put(1,1)`, `put(2,2)`, `get(1)` is 1, `put(3,3)` evicts 2, `get(2)` is -1, `get(3)` is 3, `put(4,4)` evicts 1 (tied frequency, older), `get(1)` is -1.

Approach: Map key to value, map key to frequency, and map frequency to an `OrderedDict` of keys (oldest first). Track `min_freq`.

python

```
from collections import defaultdict, OrderedDict

class LFUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.min_freq = 0
        self.kv = {}                            # key -> value
        self.kf = {}                            # key -> frequency
        self.fk = defaultdict(OrderedDict)      # freq -> keys, LRU first

    def _touch(self, key):
        f = self.kf[key]
        del self.fk[f][key]
        if not self.fk[f]:
            del self.fk[f]
            if self.min_freq == f:
                self.min_freq += 1
        self.kf[key] = f + 1
        self.fk[f + 1][key] = None

    def get(self, key):
        if key not in self.kv:
            return -1
        self._touch(key)
        return self.kv[key]

    def put(self, key, value):
        if self.cap == 0:
            return
        if key in self.kv:
            self.kv[key] = value
            self._touch(key)
            return
        if len(self.kv) == self.cap:
            old, _ = self.fk[self.min_freq].popitem(last=False)
            if not self.fk[self.min_freq]:
                del self.fk[self.min_freq]
            del self.kv[old]
            del self.kf[old]
        self.kv[key] = value
        self.kf[key] = 1
```

```
            self.fk[1][key] = None
            self.min_freq = 1
```

Complexity: Time: O(1) per op | Space: O(capacity)

Q3. Insert Delete GetRandom O(1) (LeetCode 380, Medium)

Source: LeetCode 380 | Pattern: Hash map + array

Question: Design a set with `insert(val)`, `remove(val)`, and `getRandom()` (uniform over current elements), each in average O(1).

Example: `insert(1)` is True, `remove(2)` is False, `insert(2)` is True, `getRandom()` returns 1 or 2, `remove(1)` is True, `insert(2)` is False.

Approach: A list for random access plus a dict from value to index. To remove, swap the element with the last one, then pop.

python

```python
import random

class RandomizedSet:
    def __init__(self):
        self.a = []
        self.pos = {}

    def insert(self, val):
        if val in self.pos:
            return False
        self.pos[val] = len(self.a)
        self.a.append(val)
        return True

    def remove(self, val):
        if val not in self.pos:
            return False
        i, last = self.pos[val], self.a[-1]
        self.a[i] = last
        self.pos[last] = i
        self.a.pop()
        del self.pos[val]
        return True

    def getRandom(self):
        return random.choice(self.a)
```

Complexity: Time: O(1) average per op | Space: O(n)

Q4. Random Queue (+ equality, thread safety, RLE equality)

Source: Confluent | Pattern: Array with swap-with-last

Question: A queue where `dequeue()` removes a **uniformly random** element. Part A: `enqueue`, `dequeue`, `peek`, `size`. Part B: equality as **multiset** equality (same values and counts, order ignored). Part C: thread safety. Part D: compare two run-length-encoded queues without expanding them.

Example: `[a,a,b] == [b,a,a]` is True, `[a,a,b] == [a,b,b]` is False.

Approach:

1. Store items in a list. `dequeue`: pick a random index, swap with the last element, `pop()` (no shifting).
2. Equality: snapshot each queue into a `Counter` and compare.
3. Thread safety: one lock around every method so choose-and-remove is atomic. Use `RLock` if methods call each other.
4. RLE equality: add up counts per value across all runs of each queue, then compare the two counters.

python

```
import random, threading
from collections import Counter

class RandomQueue:
    def __init__(self):
        self.a, self.lock = [], threading.RLock()

    def enqueue(self, x):
        with self.lock:
            self.a.append(x)

    def dequeue(self):
        with self.lock:
            if not self.a:
                raise IndexError("dequeue from empty queue")
            i = random.randrange(len(self.a))
            self.a[i], self.a[-1] = self.a[-1], self.a[i]
            return self.a.pop()

    def peek(self):
        with self.lock:
            if not self.a:
                raise IndexError("peek from empty queue")
            return random.choice(self.a)

    def size(self):
        with self.lock:
            return len(self.a)

    def __eq__(self, other):
        if not isinstance(other, RandomQueue):
            return NotImplemented
        with self.lock:
            mine = Counter(self.a)                    # snapshot, then release the l
        with other.lock:
            theirs = Counter(other.a)
        return mine == theirs


def rle_equal(runs1, runs2):                           # runs = [(value, count), ..
    def total(runs):
```

```
        c = Counter()
        for v, n in runs:
            c[v] += n
        return c
    return total(runs1) == total(runs2)
```

Complexity: Time: enqueue/dequeue O(1) expected, equality O(n), RLE equality O(R+V) | Space: O(n)

Watch out: Take the two locks one after the other (not nested) so two threads comparing `a == b` and `b == a` cannot deadlock. Define behavior for an empty queue (here: raise).

Q5. Time Based Key-Value Store (LeetCode 981, Medium)

Source: LeetCode 981 | Pattern: Hash map + binary search

Question: `set(key, value, timestamp)` stores a value. `get(key, timestamp)` returns the value stored with the largest timestamp `<= timestamp`, or `""` if none exists. Timestamps passed to `set` are strictly increasing.

Example: `set("foo","bar",1)`, `get("foo",1)` is `"bar"`, `get("foo",3)` is `"bar"`, `set("foo","bar2",4)`, `get("foo",4)` is `"bar2"`, `get("foo",5)` is `"bar2"`.

Approach: Per key, keep parallel sorted lists of timestamps and values. Use binary search (`bisect_right`) for `get`.

python

```python
from collections import defaultdict
import bisect

class TimeMap:
    def __init__(self):
        self.ts = defaultdict(list)
        self.vals = defaultdict(list)

    def set(self, key, value, timestamp):
        self.ts[key].append(timestamp)
        self.vals[key].append(value)

    def get(self, key, timestamp):
        if key not in self.ts:
            return ""
        i = bisect.bisect_right(self.ts[key], timestamp)
        return self.vals[key][i - 1] if i else ""
```

Complexity: Time: set O(1); get O(log n) | Space: O(total entries)

Q6. Key-Value Store with Get, GetAverage and GetMax

Source: Confluent | Pattern: Dict + running sum + lazy-deletion max-heap

Question: `put(key, value)` inserts or updates, `get(key)`, `get_average()` of all current values, `get_max()` of all current values. The catch: overwriting the current max must reveal the next max.

Example: `put(A,5)`, `put(B,8)`: max 8, average 6.5. `put(B,2)`: max 5, average 3.5.

Approach:

1. `kv` dict plus `total` (running sum). Average is `total / len(kv)`.
2. Max-heap of `(-value, key)`. Every `put` pushes a new entry and never removes the old one.
3. `get_max`: while the top entry is **stale** (`kv[key] != value`), pop it. The first valid top is the max.

python

```python
import heapq

class KVStore:
    def __init__(self):
        self.kv, self.heap, self.total = {}, [], 0

    def put(self, key, value):
        self.total += value - self.kv.get(key, 0)
        self.kv[key] = value
        heapq.heappush(self.heap, (-value, key))

    def get(self, key):
        return self.kv.get(key)

    def get_average(self):
        return self.total / len(self.kv) if self.kv else None

    def get_max(self):
        while self.heap and self.kv.get(self.heap[0][1]) != -self.heap[0][0]:
            heapq.heappop(self.heap)                        # drop stale entries
        return -self.heap[0][0] if self.heap else None
```

Complexity: Time: put O(log n), get O(1), average O(1), max amortized O(log n) | Space: O(n + updates)

Watch out: True worst-case O(1) for `get_max` under arbitrary updates is not achievable with comparison-based structures. Say so and ask whether values are bounded (then a bucket approach works).

Q7. Design Authentication Manager (LeetCode 1797, Medium)

Source: LeetCode 1797 | Pattern: Hash map with expiry

Question: Tokens live `timeToLive` seconds. `generate(id, t)` creates a token expiring at `t + ttl`. `renew(id, t)` extends it only if still unexpired. `countUnexpiredTokens(t)` returns how many are alive. A token expiring at time `t` counts as expired at `t`.

Example: ttl=5: `generate("aaa",2)`, `count(6)` is 1, `generate("bbb",7)`, `renew("aaa",8)` ignored since it expired at 7, `renew("bbb",10)`, `count(15)` is 0.

Approach: Map token to expiry time. Alive means `expiry > currentTime`.

python

```python
class AuthenticationManager:
    def __init__(self, timeToLive):
        self.ttl = timeToLive
        self.exp = {}

    def generate(self, tokenId, currentTime):
        self.exp[tokenId] = currentTime + self.ttl

    def renew(self, tokenId, currentTime):
        if self.exp.get(tokenId, 0) > currentTime:
            self.exp[tokenId] = currentTime + self.ttl

    def countUnexpiredTokens(self, currentTime):
        return sum(e > currentTime for e in self.exp.values())
```

Complexity: Time: generate / renew O(1); count O(n) | Space: O(n)

Note: For amortized O(1) `count`, keep an `OrderedDict` ordered by expiry and pop expired entries from the front.

Q8. Token Manager: generate / renew / list active tokens by create_time

Source: Confluent ("Retrieve Token List") | Pattern: Dict + sort on read

Question: Tokens have a TTL. `generate(id, t)` registers a token, `renew(id, t)` resets its expiry only if it has not expired, and `retrieve(t)` returns the active tokens ordered by `create_time` (tie-break by id). Active means `now < expiry`. `create_time` may repeat.

Example: ttl=10: `generate(A,100)`, `generate(B,105)`, `generate(C,100)`. `retrieve(108)` is `[A, C, B]`. `renew(A,108)` makes A expire at 118. `retrieve(112)` is `[A, B]` (B expires at 115, C at 110).

Approach:

1. `tok[id] = [create_time, expiry]`.
2. `renew`: fail if missing or `expiry <= now`; else `expiry = now + ttl` (create_time unchanged, so the order does not change).
3. `retrieve`: filter `expiry > now`, sort by `(create_time, id)`.
4. If `retrieve` is called very often, keep a `SortedList` keyed by `(create_time, id)` (needs `sortedcontainers`, not stdlib).

python

```python
class TokenManager:
    def __init__(self, ttl):
        self.ttl = ttl
        self.tok = {}                                   # id -> [create_time, exp

    def generate(self, tid, now):
        if tid in self.tok:
            return False
        self.tok[tid] = [now, now + self.ttl]
        return True

    def renew(self, tid, now):
        t = self.tok.get(tid)
        if not t or t[1] <= now:
            return False
        t[1] = now + self.ttl
        return True

    def retrieve(self, now):
        live = [(c, i) for i, (c, e) in self.tok.items() if e > now]
        return [i for _, i in sorted(live)]
```

Complexity: Time: generate/renew O(1), list O(n log n) | Space: O(n)

Watch out: Expired means `expiry <= now`. Optionally delete expired entries while scanning to keep memory bounded.

Q9. Sliding-Window Cache: put / get / average

Source: Confluent qualifier (senior) | Pattern: OrderedDict + running sum

Question: Build a store with a time window. `put(key, value, t)`, `get(key, t)` (returns `None` if expired), and `average(t)` over unexpired entries. Anything older than the window is evicted, including when computing the average.

Example: window = 1 hour: `put(A,10)@00:00`, `put(B,20)@00:10`, `average@00:30` is `15`, `average@01:05` is `20` (A expired), `get(B)@01:08` is `20`, `put(A,30)@01:15`, `average@01:50` is `30`.

Approach:

1. `OrderedDict` key -> `(value, timestamp)`, oldest first. Time never goes backwards, so insertion order is timestamp order.

2. Keep `total` (sum of live values).
3. Every operation first evicts from the front while `now - ts >= window`, subtracting from `total`.
4. `put` on an existing key: pop it, subtract its old value, re-insert at the end.

python

```python
from collections import OrderedDict

class WindowCache:
    def __init__(self, window):
        self.window, self.d, self.total = window, OrderedDict(), 0   # key -> (val

    def _evict(self, now):
        while self.d:
            _, (v, ts) = next(iter(self.d.items()))
            if now - ts < self.window:
                break
            self.d.popitem(last=False)
            self.total -= v

    def put(self, key, value, now):
        self._evict(now)
        if key in self.d:
            self.total -= self.d.pop(key)[0]
        self.d[key] = (value, now)
        self.total += value

    def get(self, key, now):
        self._evict(now)
        return self.d[key][0] if key in self.d else None

    def average(self, now):
        self._evict(now)
        return self.total / len(self.d) if self.d else 0.0
```

Complexity: Time: O(1) amortized per op | Space: O(n)

Watch out: Clarify the boundary (`>= window` expired vs `> window`) and what the average of an empty window is.

Q10. Message Logger (10-Second Rate Limit per Message)

Source: Confluent | Pattern: Hash map of last printed time

Question: `shouldPrintMessage(timestamp, message)` returns `True` if the message may be printed, `False` if the same message was printed less than 10 seconds ago. Messages are independent. Timestamps are non-decreasing.

Example: `(0,"order")` True, `(1,"order")` False, `(2,"ship")` True, `(11,"order")` True, `(10,"x")` after `(0,"x")` is True (10 - 0 >= 10).

Approach:

1. `last[message]` = timestamp of the last **printed** time.
2. If it exists and `timestamp - last < 10`: return `False` (do **not** update).
3. Otherwise set `last[message] = timestamp`, return `True`.
4. Follow-ups: memory cleanup uses a queue of `(ts, msg)`, and thread safety uses a lock (second class).

python

```python
class Logger:
    def __init__(self, window=10):
        self.window, self.last = window, {}

    def shouldPrintMessage(self, timestamp, message):
        if message in self.last and timestamp - self.last[message] < self.window:
            return False
        self.last[message] = timestamp
        return True


import threading
from collections import deque

class SafeLogger:                                   # bounded memory + thread saf
    def __init__(self, window=10):
        self.window, self.q, self.live, self.lock = window, deque(), set(), threa

    def shouldPrintMessage(self, timestamp, message):
        with self.lock:
            while self.q and timestamp - self.q[0][0] >= self.window:
                self.live.discard(self.q.popleft()[1])       # forget expired mess
            if message in self.live:
                return False
            self.q.append((timestamp, message))
            self.live.add(message)
            return True
```

Complexity: Time: O(1) per call | Space: O(M) distinct messages

Watch out: The queue version needs non-decreasing timestamps. For many servers, use a shared store (e.g. Redis with expiring keys).

Q11. Silent Sensor Detector (STABLE / UNSTABLE)

Source: Confluent | Pattern: Per-sensor sorted list + binary search

Question: Record pings `(sensor, timestamp)` and answer whether a sensor is `STABLE` or `UNSTABLE` at query time `T`. The rule is not specified in the original, so **clarify it**. Assumed here: STABLE if at least one ping lies in `[T - K, T]`, unknown sensor means UNSTABLE.

Example: `K=10`, pings `100, 115, 105` for `S1`. `T=108` STABLE (105), `T=120` STABLE (115), `T=130` UNSTABLE, `T=90` UNSTABLE, unknown sensor UNSTABLE.

Approach:

1. `pings[sensor]` = sorted list (`bisect.insort`, handles out-of-order pings).
2. Query: `i = bisect_right(list, T)`; the latest ping at or before `T` is `list[i-1]`.
3. STABLE if `i > 0` and that ping `>= T - K`.
4. If pings are in order and queries are "now" only, store just the latest timestamp (O(1) space per sensor).

python

```python
import bisect
from collections import defaultdict

class SensorHealth:
    def __init__(self, k=10):
        self.k, self.pings = k, defaultdict(list)

    def record_ping(self, sensor, ts):
        bisect.insort(self.pings[sensor], ts)

    def status(self, sensor, T):
        p = self.pings.get(sensor)
        if not p:
            return "UNSTABLE"
        i = bisect.bisect_right(p, T)
        return "STABLE" if i and p[i - 1] >= T - self.k else "UNSTABLE"
```

Complexity: Time: record O(P) worst (O(1) if in order), query O(log P) | Space: O(P)

Watch out: Questions to ask: exact rule, inclusive boundary, out-of-order pings, can `T` go backwards, unknown sensors, scale and memory limits.

Q12. Cache With Time Limit (LeetCode 2622, Medium; JS-only on LeetCode, Python port)

Source: LeetCode 2622 | Pattern: Hash map with expiry

Question: Build a cache where each key expires after a duration. `set(key, value, duration)` returns `true` if an unexpired key already existed (and overwrites value and

duration). `get(key)` returns the value or `-1`. `count()` returns the number of unexpired keys.

Example: `set(1,42,100)` is false, `get(1)` at 50 ms is 42, `count()` at 50 ms is 1, `get(1)` at 150 ms is -1.

Python (lazy expiry, no timers):

```python
import time

class TimeLimitedCache:
    def __init__(self):
        self.d = {}                                     # key -> (value, expiry)

    def set(self, key, value, duration_ms):
        now = time.monotonic()
        existed = key in self.d and self.d[key][1] > now
        self.d[key] = (value, now + duration_ms / 1000)
        return existed

    def get(self, key):
        item = self.d.get(key)
        if item and item[1] > time.monotonic():
            return item[0]
        self.d.pop(key, None)
        return -1

    def count(self):
        now = time.monotonic()
        self.d = {k: v for k, v in self.d.items() if v[1] > now}
        return len(self.d)
```

Complexity: Time: set / get O(1); count O(n) with lazy expiry | Space: O(n)

Q13. Debounce (LeetCode 2627, Medium; JS-only on LeetCode, Python port)

Source: LeetCode 2627 | Pattern: Timer

Question: Return a debounced version of `fn`. Each call delays execution by `t` ms, and a new call before the delay ends cancels the pending one.

Example: `t=50`, calls at 50 ms with `[1]` and 75 ms with `[2]`. Only `fn(2)` runs, at 125 ms.

Python:

```python
import threading

def debounce(fn, t_seconds):
    timer = None
    lock = threading.Lock()

    def wrapper(*args, **kwargs):
        nonlocal timer
        with lock:
            if timer is not None:
                timer.cancel()
            timer = threading.Timer(t_seconds, fn, args, kwargs)
            timer.start()
    return wrapper
```

Complexity: Time: O(1) per call | Space: O(1)

Q14. Promise Time Limit (LeetCode 2637, Medium; JS-only on LeetCode, Python port)

Source: LeetCode 2637 | Pattern: Async timeout

Question: Given an async function `fn` and a time limit `t` ms, return a new function that resolves with `fn`'s result if it finishes within `t` ms. Otherwise it rejects with `"Time Limit Exceeded"`. If `fn` rejects first, reject with that error.

Example: `fn` takes 100 ms, `t=50`, so it rejects with `"Time Limit Exceeded"`. With `t=150` it resolves normally.

Python:

```python
```

```python
import asyncio

def time_limit(fn, t_ms):
    async def wrapper(*args, **kwargs):
        try:
            return await asyncio.wait_for(fn(*args, **kwargs), timeout=t_ms / 1000
        except asyncio.TimeoutError:
            raise Exception("Time Limit Exceeded")
    return wrapper
```

Complexity: Time: O(1) overhead per call | Space: O(1)

Part 2: Time Windows, Intervals and Heaps

Q15. Count Events in the Window (t − W, t] for Each Event

Source: Confluent / PracHub | Pattern: Two pointers on a sorted array

Question: Given sorted event timestamps, for each event return how many events fall in `(t - W, t]`. Can you do it in O(n)?

Example: `[1,2,4,7], W=3` gives `[1,2,2,1]`. `[1,1,2,3], W=1` gives `[2,2,1,1]`.

Approach:

1. `hi` = first index with `ts[hi] > t` (so equal timestamps are all counted).
2. `lo` = first index with `ts[lo] > t - W`.
3. Answer for this event is `hi - lo`. Both pointers only move forward, so it is O(n).

```python
def count_events(ts, W):
    n, lo, hi, res = len(ts), 0, 0, []
    for t in ts:
        while hi < n and ts[hi] <= t:        # include events with the same timesta
            hi += 1
        while ts[lo] <= t - W:               # drop events outside (t-W, t]
            lo += 1
        res.append(hi - lo)
    return res
```

Complexity: Time: O(n) | Space: O(1) extra

Watch out: The window is open on the left, so use `<=` when dropping. Equivalent one-liner per event: `bisect_right(ts, t) - bisect_right(ts, t - W)` in O(n log n).

Q16. Meeting Rooms II (LeetCode 253, Medium)

Source: LeetCode 253 | Pattern: Sort + min-heap

Question: Given meeting intervals `[start, end]`, return the minimum number of rooms needed. A meeting ending at `t` and another starting at `t` can share a room.

Example: `[[0,30],[5,10],[15,20]]` gives `2`. `[[7,10],[2,4]]` gives `1`.

Approach: Sort by start. A min-heap holds the end times of rooms in use. If the earliest end is `<= start`, reuse that room.

```python
import heapq

def minMeetingRooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []
    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heappop(heap)
        heapq.heappush(heap, end)
    return len(heap)
```

Complexity: Time: O(n log n) | Space: O(n)

Q17. Meeting Rooms III (LeetCode 2402, Hard)

Source: LeetCode 2402 | Pattern: Two heaps

Question: There are `n` rooms numbered `0..n-1` and meetings `[start, end]` with unique start times. Each meeting takes the lowest-numbered free room. If none is free, it waits for the first room to free up and keeps its original duration. Earlier original start times get priority. Return the room that hosted the most meetings (lowest number on ties).

Example: `n=2, meetings=[[0,10],[1,5],[2,7],[3,4]]` gives `0`.

Approach: Two heaps: free rooms (by id) and busy rooms (by `(end_time, room)`). Before each meeting, release rooms that have finished. If none is free, take the room with the earliest end and push its end time forward by the meeting's duration.

```python
import heapq

def mostBooked(n, meetings):
    meetings.sort()
    free = list(range(n))
    heapq.heapify(free)
    busy = []                          # (end_time, room)
    cnt = [0] * n

    for s, e in meetings:
        while busy and busy[0][0] <= s:
            _, r = heapq.heappop(busy)
            heapq.heappush(free, r)
        if free:
            r = heapq.heappop(free)
            heapq.heappush(busy, (e, r))
        else:
            end, r = heapq.heappop(busy)
            heapq.heappush(busy, (end + (e - s), r))   # delayed start
        cnt[r] += 1

    return cnt.index(max(cnt))      # index() returns the lowest room on ties
```

Complexity: Time: O(m log m + m log n) | Space: O(n + m)

Q18. Meeting Scheduler (LeetCode 1229, Medium)

Source: LeetCode 1229 | Pattern: Sort + two pointers

Question: Given two people's availability slots and a `duration`, return the earliest `[start, start + duration]` that fits in a slot free for both, or `[]` if none exists.

Example: `slots1=[[10,50],[60,120],[140,210]]`, `slots2=[[0,15],[60,70]]`, `duration=8` gives `[60,68]`. With `duration=12` it gives `[]`.

Approach: Sort both lists and use two pointers. The overlap is `[max(starts), min(ends)]`. If it fits, return it. Otherwise advance the pointer whose slot ends first.

```python
```

```python
def minAvailableDuration(slots1, slots2, duration):
    slots1.sort(); slots2.sort()
    i = j = 0
    while i < len(slots1) and j < len(slots2):
        start = max(slots1[i][0], slots2[j][0])
        end = min(slots1[i][1], slots2[j][1])
        if end - start >= duration:
            return [start, start + duration]
        if slots1[i][1] < slots2[j][1]:
            i += 1
        else:
            j += 1
    return []
```

Complexity: Time: O(n log n + m log m) | Space: O(1) extra

Q19. Kth Largest Element in an Array (LeetCode 215, Medium)

Source: LeetCode 215 | Pattern: Min-heap of size k

Question: Return the k-th largest element of `nums` (by sorted order, not distinct values).

Example: `[3,2,1,5,6,4], k=2` gives `5`.

Approach: Keep a min-heap of the `k` largest values seen. Its root is the answer. (Quickselect gives O(n) average.)

```python
import heapq

def findKthLargest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)
    for x in nums[k:]:
        if x > heap[0]:
            heapq.heapreplace(heap, x)
    return heap[0]
```

Complexity: Time: O(n log k) | Space: O(k)

Note: Quickselect gives O(n) average.

Q20. Top K Frequent Elements (LeetCode 347, Medium)

Source: LeetCode 347 | Pattern: Counter + bucket sort

Question: Return the `k` most frequent elements of `nums`, in any order.

Example: `[1,1,1,2,2,3], k=2` gives `[1,2]`.

Approach: Count with `Counter`, then bucket by frequency and read from the highest bucket down. This is O(n) and beats a heap.

```python
from collections import Counter

def topKFrequent(nums, k):
    cnt = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for x, c in cnt.items():
        buckets[c].append(x)
    res = []
    for c in range(len(buckets) - 1, 0, -1):
        for x in buckets[c]:
            res.append(x)
            if len(res) == k:
                return res
```

One-liner alternative: `[x for x, _ in Counter(nums).most_common(k)]` (O(n log k)).

Complexity: Time: O(n) | Space: O(n)

Note: One-liner: `[x for x, _ in Counter(nums).most_common(k)]` (O(n log k)).

Q21. Find Median from Data Stream (LeetCode 295, Hard)

Source: LeetCode 295 | Pattern: Two heaps

Question: Design a structure with `addNum(num)` and `findMedian()` for a stream of integers.

Example: `addNum(1)`, `addNum(2)`, `findMedian()` is `1.5`, `addNum(3)`, `findMedian()` is `2.0`.

Approach: Two heaps. `lo` is a max-heap (stored negated) with the smaller half, and `hi` is a min-heap with the larger half. Keep `len(lo)` equal to `len(hi)` or one greater.

```python
```

```python
import heapq

class MedianFinder:
    def __init__(self):
        self.lo = []    # max-heap via negation
        self.hi = []    # min-heap

    def addNum(self, num):
        heapq.heappush(self.lo, -num)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))

    def findMedian(self):
        if len(self.lo) > len(self.hi):
            return float(-self.lo[0])
        return (-self.lo[0] + self.hi[0]) / 2
```

Complexity: Time: addNum O(log n); findMedian O(1) | Space: O(n)

Q22. Merge k Sorted Lists (LeetCode 23, Hard)

Source: LeetCode 23 | Pattern: Min-heap

Question: Merge `k` sorted linked lists into one sorted linked list.

Example: `[[1,4,5],[1,3,4],[2,6]]` gives `[1,1,2,3,4,4,5,6]`.

Approach: A min-heap holds the current head of each list. The index in the tuple breaks ties so nodes are never compared.

python

```python
import heapq

def mergeKLists(lists):
    heap = [(node.val, i, node) for i, node in enumerate(lists) if node]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

Complexity: Time: O(N log k), N = total nodes | Space: O(k)

Q23. Minimum Health for Gaming (rank-th strongest each level)

Source: Confluent (reconstructed) | Pattern: Min-heap of size `rank`

Question (reconstructed from a vague write-up): Start with `initial_players` (strengths). Each level adds one player from `next_players`. At every level you defeat the player ranked `rank` by strength (rank 1 = strongest), losing health equal to that strength. Return the minimum starting health so that health is still `>= 0` at the end, which is the sum of all defeated strengths. **Confirm** whether the first level uses only the initial players and what happens when fewer than `rank` players exist (assumed: no fight that level).

Example: `initial=[3,1,5]`, `next=[2,10]`, `rank=2`: level 1 opponent is 3, level 2 (adds 2) is 3, level 3 (adds 10) is 5. Total `11`.

Approach:

1. Keep a **min-heap that holds only the `rank` strongest players**. If it grows beyond `rank`, pop the smallest.
2. When the heap has exactly `rank` players, its root is the rank-th strongest.
3. Add the root to the total at each level.

```python
```

```python
import heapq

def min_health(initial_players, next_players, rank):
    heap, total = [], 0

    def add(x):
        heapq.heappush(heap, x)
        if len(heap) > rank:
            heapq.heappop(heap)

    for x in initial_players:
        add(x)
    if len(heap) == rank:
        total += heap[0]
    for x in next_players:
        add(x)
        if len(heap) == rank:
            total += heap[0]
    return total
```

Complexity: Time: O((n + L) log rank) | Space: O(rank)

Part 3: Text, Files and Search

Q24. Search a Word or Phrase in Many Documents (+ AND / OR / brackets)

Source: Confluent onsite | Pattern: Inverted index with positions

Question: Given documents `(id, text)`, return the ids of documents containing a word, or containing a phrase with the words adjacent and in order. Must be efficient for 1M+ documents. Follow-up: support queries like `cloud AND (monitoring OR computing)`.

Example: docs 1-3 from the prompt: `search("cloud")` is `[1,2,3]`, `search("cloud monitoring")` is `[2]`, `search("Cloud computing is")` is `[1,3]`.

Approach:

1. Tokenize: lowercase, `\w+`.
2. Index: `word -> doc_id -> [positions]`.
3. Phrase: intersect the doc sets of all words, then check `pos, pos+1, pos+2...` exist for the words in that doc.
4. Follow-up: tiny recursive-descent parser. `OR` has lowest precedence, then `AND`, then `( )` or a term. Each term returns a set of doc ids, then use `|` and `&`.

python

```
import re
from collections import defaultdict

class PhraseSearch:
    def __init__(self):
        self.idx = defaultdict(lambda: defaultdict(list))   # word -> doc_id -> [p

    @staticmethod
    def tok(text):
        return re.findall(r"\w+", text.lower())

    def add(self, doc_id, text):
        for pos, w in enumerate(self.tok(text)):
            self.idx[w][doc_id].append(pos)

    def search(self, phrase):
        words = self.tok(phrase)
        if not words or any(w not in self.idx for w in words):
            return []
        docs = set(self.idx[words[0]])
        for w in words[1:]:
            docs &= set(self.idx[w])                # docs containing every word
        res = []
        for d in docs:
            pos_sets = [set(self.idx[w][d]) for w in words]
            if any(all(p + i in pos_sets[i] for i in range(1, len(words)))
                   for p in self.idx[words[0]][d]):
                res.append(d)
        return sorted(res)

    def query(self, q):                             # follow-up: AND / OR / ( )
        toks, pos = re.findall(r'\(|\)|"[^"]+"|\w+', q), 0

        def parse_or():
            nonlocal pos
            res = parse_and()
            while pos < len(toks) and toks[pos] == "OR":
                pos += 1
                res = res | parse_and()
            return res
```

```
        def parse_and():
            nonlocal pos
            res = atom()
            while pos < len(toks) and toks[pos] == "AND":
                pos += 1
                res = res & atom()
            return res

        def atom():
            nonlocal pos
            t = toks[pos]; pos += 1
            if t == "(":
                res = parse_or(); pos += 1           # skip ")"
                return res
            return set(self.search(t.strip('"')))

        return sorted(parse_or())
```

Complexity: Time: build O(T), query about O(postings of the query words) | Space: O(T)

Watch out: Use `\w+` for tokens so punctuation never breaks a phrase. Quote multi-word terms in the boolean query, e.g. `"cloud computing" AND monitor`. At real scale, shard the index by doc id and merge results.

Q25. Text Search Engine: WORD and PHRASE Counts

Source: Confluent (PracHub) | Pattern: Counter + sliding window

Question: Tokenize a multi-line text (split on non-alphanumerics, case-insensitive). Answer `WORD w` (how many tokens equal `w`) and `PHRASE p` (how many times the phrase's tokens appear consecutively, in order).

Example: Text `Hello, world! world: hello.` gives tokens `hello world world hello`. `WORD world` is `2`, `PHRASE hello world` is `1`, `PHRASE world hello` is `1`.

Approach:

1. Tokenize once and keep a `Counter` for words.
2. For a phrase, tokenize it to `q` of length `k`, then compare every window `tokens[i:i+k] == q`.
3. Keep tokens continuous across line breaks unless told otherwise (ask).
4. If there are many phrase queries, switch to the positional index from the previous question.

python

```python
import re
from collections import Counter

class TextSearch:
    def __init__(self, text):
        self.tokens = re.findall(r"[a-z0-9]+", text.lower())
        self.freq = Counter(self.tokens)

    def word(self, w):
        return self.freq[w.lower()]

    def phrase(self, p):
        q = re.findall(r"[a-z0-9]+", p.lower())
        k = len(q)
        if k == 0:
            return 0
        return sum(self.tokens[i:i + k] == q for i in range(len(self.tokens) - k +
```

Complexity: Time: WORD O(1), PHRASE O(T·P) per query | Space: O(T)

Q26. Unix `tail -N` (Last N Lines of a Huge File)

Source: Confluent onsite | Pattern: Read blocks backwards / bounded deque

Question: Print the last `N` lines of a very large file in original order without reading the whole file. Allowed API in the original: `read(k)`, `movePointer(pos)`, `getSize()`.

Example: A file with lines `1..100` and `N=3` returns lines `98, 99, 100`.

Approach:

1. `N <= 0`: return nothing.
2. Seek to the end. Read fixed-size blocks **backwards**, prepending each to a buffer.
3. Stop when the buffer holds **more than N newlines** (so the first kept line is complete) or you reach the start.
4. Split on `b"\n"`, drop the empty tail caused by a trailing newline, keep the last N, decode.
5. No seeking (stdin or pipe): `deque(lines, maxlen=N)` in one pass.

python

```python
from collections import deque

def tail(path, n, block=8192):
    if n <= 0:
        return []
    with open(path, "rb") as f:
        f.seek(0, 2)
        pos = f.tell()                          # getSize()
        data, newlines = b"", 0
        while pos > 0 and newlines <= n:        # need > n newlines
            step = min(block, pos)
            pos -= step
            f.seek(pos)                         # movePointer(pos)
            chunk = f.read(step)                # read(k)
            data = chunk + data
            newlines += chunk.count(b"\n")
        lines = data.split(b"\n")
        if lines and lines[-1] == b"":          # trailing newline
            lines.pop()
        return [l.decode("utf-8", "replace") for l in lines[-n:]]

def tail_stream(lines, n):
    return list(deque(lines, maxlen=n)) if n > 0 else []
```

Complexity: Time: O(size of the last N lines), not O(file) | Space: O(N lines + one block)

Watch out: Work in bytes (`\n` never appears inside a multi-byte UTF-8 character), then decode. Block size trade-off: a tiny block means many seeks, a huge block wastes reads, so use 4-64 KB. Threads do not help (sequential, I/O-bound, GIL).

Q27. Function Overload Matching (with Variadic Parameters)

Source: Confluent onsite | Pattern: Rule-based scan

Question: Register functions `(name, param_types, is_card)`. Given a list of argument types, return **all** registered functions that can be called with them. `is_card=False`: arg count and types must match exactly. `is_card=True`: the **last** parameter type may repeat zero or more times.

Example: `f1(Integer, Boolean)` matches `[Integer, Boolean]` only. `f2(Integer…)` (variadic) matches `[]`, `[Integer]`, `[Integer, Integer, Integer]`, not `[String]`.

Approach:

1. Exact: `list(params) == list(args)`.
2. Variadic: `fixed = params[:-1]`, `var = params[-1]`. Need `len(args) >= len(fixed)`, `args[:len(fixed)] == fixed`, and every remaining arg equals `var`.
3. Return every match, do not stop at the first and do not dedupe.
4. Variadic with empty `params`: ask. Here it is rejected at registration.

python

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Function:
    name: str
    params: tuple
    is_card: bool = False

class FunctionRegistry:
    def __init__(self):
        self.functions = []

    def register(self, name, params, is_card=False):
        if is_card and not params:
            raise ValueError("variadic function needs at least one parameter type")
        self.functions.append(Function(name, tuple(params), is_card))

    def match(self, args):
        return [f for f in self.functions if self._matches(f, tuple(args))]

    @staticmethod
    def _matches(f, args):
        if not f.is_card:
            return f.params == args
        fixed, var = f.params[:-1], f.params[-1]
        return (len(args) >= len(fixed)
                and args[:len(fixed)] == fixed
                and all(a == var for a in args[len(fixed):]))
```

Complexity: Time: O(N·M) per lookup | Space: O(N·M)

Watch out: Faster version for many functions: hash exact signatures in a dict by `params`, and index variadic functions by `(fixed_prefix, var_type)`. Lookup then does not depend on N.

Q28. Wildcard Matching (LC 44) and Regular Expression Matching (LC 10)

Source: Glassdoor (Confluent reports "pattern matching") | Pattern: 2D DP

Question: Given string `s` and pattern `p`, decide whether the **whole** string matches. Wildcard: `?` is any one character, `*` is any sequence (including empty). Regex: `.` is any one character, `x*` is zero or more of the previous character.

Example: Wildcard `("adceb","*a*b")` is True, `("acdcb","a*c?b")` is False. Regex `("aa","a*")` is True, `("mississippi","mis*is*p*.")` is False.

Approach:

1. `dp[i][j]` = first `i` chars of `s` match first `j` chars of `p`. `dp[0][0] = True`.
2. Wildcard `*`: `dp[i][j] = dp[i][j-1]` (empty) or `dp[i-1][j]` (consume one char).
3. Regex `x*`: `dp[i][j] = dp[i][j-2]` (zero copies) or (`p[j-2]` matches `s[i-1]` and `dp[i-1][j]`).
4. Otherwise a single-character match is `dp[i][j] = dp[i-1][j-1]` when the characters match (or `?` / `.`).
5. First row: leading `*` (or `x*` in regex) can match the empty string.

python

```python
def isMatchWildcard(s, p):
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
    for j in range(1, n + 1):
        if p[j - 1] == "*":
            dp[0][j] = dp[0][j - 1]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if p[j - 1] == "*":
                dp[i][j] = dp[i][j - 1] or dp[i - 1][j]
            elif p[j - 1] in ("?", s[i - 1]):
                dp[i][j] = dp[i - 1][j - 1]
    return dp[m][n]


def isMatchRegex(s, p):
    m, n = len(s), len(p)
    dp = [[False] * (n + 1) for _ in range(m + 1)]
    dp[0][0] = True
    for j in range(2, n + 1):
        if p[j - 1] == "*":
            dp[0][j] = dp[0][j - 2]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if p[j - 1] == "*":
                dp[i][j] = dp[i][j - 2] or (p[j - 2] in (s[i - 1], ".") and dp[i
            elif p[j - 1] in (s[i - 1], "."):
                dp[i][j] = dp[i - 1][j - 1]
    return dp[m][n]
```

Complexity: Time: O(m·n) | Space: O(m·n)

Watch out: These two are only mentioned in the Glassdoor note of your file. I added the standard solutions so you have them ready.

Q29. Design Add and Search Words Data Structure (LeetCode 211, Medium)

Source: LeetCode 211 | Pattern: Trie

Question: Support `addWord(word)` and `search(word)`, where `search` may contain `.` that matches any single letter.

Example: Add `bad`, `dad`, `mad`. `search("pad")` is False, `search("bad")` is True, `search(".ad")` is True, `search("b..")` is True.

Approach: A trie. On `.`, try every child with DFS.

```python
class WordDictionary:
    def __init__(self):
        self.root = {}

    def addWord(self, word):
        node = self.root
        for ch in word:
            node = node.setdefault(ch, {})
        node['$'] = True

    def search(self, word):
        def dfs(i, node):
            if i == len(word):
                return '$' in node
            ch = word[i]
            if ch == '.':
                return any(dfs(i + 1, child) for k, child in node.items() if k !=
            return ch in node and dfs(i + 1, node[ch])
        return dfs(0, self.root)
```

Complexity: Time: add O(L); search O(L), worst O(26^L) with dots | Space: O(total characters)

Q30. Word Search II (LeetCode 212, Hard)

Source: LeetCode 212 | Pattern: Trie + DFS

Question: Given a grid of letters and a list of words, return all words that can be formed in the grid (same adjacency rules as Word Search, no cell reuse within a word).

Example: `board=[["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]]`, `words=["oath","pea","eat","rain"]` gives `["eat","oath"]`.

Approach: Build a trie of all words and run one DFS per start cell, walking the trie at the same time. Store the full word at its end node, and prune exhausted branches to avoid repeated work.

```python
def findWords(board, words):
    root = {}
    for w in words:
        node = root
        for ch in w:
            node = node.setdefault(ch, {})
        node['$'] = w

    R, C = len(board), len(board[0])
    res = []

    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent[ch]
        if '$' in node:
            res.append(node.pop('$'))        # pop avoids duplicates
        board[r][c] = '#'
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and board[nr][nc] in node:
                dfs(nr, nc, node)
        board[r][c] = ch
        if not node:
            parent.pop(ch)                   # prune empty branch

    for r in range(R):
        for c in range(C):
            if board[r][c] in root:
                dfs(r, c, root)
    return res
```

Complexity: Time: O(R·C·3^L) worst case, far less with pruning | Space: O(total word characters)

Q31. Word Search (LeetCode 79, Medium)

Source: LeetCode 79 | Pattern: DFS backtracking

Question: Given a grid of letters and a word, return whether the word can be built from sequentially adjacent cells (up, down, left, right) without reusing a cell.

Example: `board=[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`. `"ABCCED"` is True, `"SEE"` is True, `"ABCB"` is False.

Approach: DFS from every cell, marking the current cell as visited (`'#'`) and restoring it on backtrack.

python

```python
def exist(board, word):
    R, C = len(board), len(board[0])

    def dfs(r, c, i):
        if i == len(word):
            return True
        if r < 0 or c < 0 or r >= R or c >= C or board[r][c] != word[i]:
            return False
        tmp, board[r][c] = board[r][c], '#'
        ok = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
              dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = tmp
        return ok

    return any(dfs(r, c, 0) for r in range(R) for c in range(C))
```

Complexity: Time: O(R·C·3^L) | Space: O(L) recursion

Part 4: Backtracking, DP, Graphs and Binary Search

Q32. Valid Sudoku (LeetCode 36, Medium)

Source: LeetCode 36 | Pattern: Hash set

Question: Given a 9x9 board with digits `1-9` or `.`, check that no filled cell repeats a digit in its row, column, or 3x3 box. The board does not need to be solvable.

Example: A board with two `5`s in the first row returns `False`. A board with no repeats returns `True`.

Approach: Keep a set per row, column, and box. Box index is `(r // 3) * 3 + c // 3`.

python

```python
def isValidSudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == '.':
                continue
            b = (r // 3) * 3 + c // 3
            if v in rows[r] or v in cols[c] or v in boxes[b]:
                return False
            rows[r].add(v); cols[c].add(v); boxes[b].add(v)
    return True
```

Complexity: Time: O(81) = O(1) | Space: O(81) = O(1)

Q33. Sudoku Solver (LeetCode 37, Hard)

Source: LeetCode 37 | Pattern: Backtracking

Question: Fill the empty cells (`.`) of a 9x9 board in place so that every row, column, and 3x3 box contains `1-9` exactly once. A unique solution is guaranteed.

Example: A standard puzzle gets completed in place, nothing is returned.

Approach: Backtracking over the empty cells with row, column, and box sets for fast validity checks. Optional speedup: always pick the empty cell with the fewest candidates.

```python
```

```python
def solveSudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
    empties = []
    for r in range(9):
        for c in range(9):
            v = board[r][c]
            if v == '.':
                empties.append((r, c))
            else:
                rows[r].add(v); cols[c].add(v); boxes[(r // 3) * 3 + c // 3].add(v

    def backtrack(i):
        if i == len(empties):
            return True
        r, c = empties[i]
        b = (r // 3) * 3 + c // 3
        for d in "123456789":
            if d in rows[r] or d in cols[c] or d in boxes[b]:
                continue
            board[r][c] = d
            rows[r].add(d); cols[c].add(d); boxes[b].add(d)
            if backtrack(i + 1):
                return True
            board[r][c] = '.'
            rows[r].remove(d); cols[c].remove(d); boxes[b].remove(d)
        return False

    backtrack(0)
```

Complexity: Time: exponential worst case, fast in practice with pruning | Space: O(81)

Q34. Combination Sum (LeetCode 39, Medium)

Source: LeetCode 39 | Pattern: Backtracking

Question: Given distinct integers `candidates` and a `target`, return all unique combinations that sum to `target`. Each number can be reused any number of times.

Example: `[2,3,6,7], target=7` gives `[[2,2,3],[7]]`.

Approach: Backtracking. Recurse with the same index `i` to allow reuse. Moving only forward avoids duplicate orderings.

```python
def combinationSum(candidates, target):
    candidates.sort()
    res, path = [], []

    def dfs(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            c = candidates[i]
            if c > remain:
                break
            path.append(c)
            dfs(i, remain - c)          # i, not i + 1: reuse allowed
            path.pop()

    dfs(0, target)
    return res
```

Complexity: Time: exponential, bounded by the number of combinations | Space: O(target / min) recursion depth

---

Q35. Combination Sum II (LeetCode 40, Medium)

Source: LeetCode 40 | Pattern: Backtracking + skip duplicates

Question: `candidates` may contain duplicates, and each element can be used at most once. Return all unique combinations summing to `target`.

Example: `[10,1,2,7,6,1,5], target=8` gives `[[1,1,6],[1,2,5],[1,7],[2,6]]`.

Approach: Sort, recurse with `i + 1`, and skip duplicates at the same depth (`i > start and c[i] == c[i-1]`).

```python
```

```python
def combinationSum2(candidates, target):
    candidates.sort()
    res, path = [], []

    def dfs(start, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue                # skip duplicate at this level
            if candidates[i] > remain:
                break
            path.append(candidates[i])
            dfs(i + 1, remain - candidates[i])
            path.pop()

    dfs(0, target)
    return res
```

Complexity: Time: O(2^n) worst case | Space: O(n)

Q36. Combination Sum III (LeetCode 216, Medium)

Source: LeetCode 216 | Pattern: Backtracking

Question: Find all combinations of exactly `k` distinct numbers from `1-9` that sum to `n`.

Example: `k=3, n=7` gives `[[1,2,4]]`. `k=3, n=9` gives `[[1,2,6],[1,3,5],[2,3,4]]`.

Approach: Backtracking over digits 1 to 9, stopping once `k` numbers are chosen.

python

```python
def combinationSum3(k, n):
    res, path = [], []

    def dfs(start, remain):
        if len(path) == k:
            if remain == 0:
                res.append(path[:])
            return
        for d in range(start, 10):
            if d > remain:
                break
            path.append(d)
            dfs(d + 1, remain - d)
            path.pop()

    dfs(1, n)
    return res
```

Complexity: Time: O(C(9,k)·k) | Space: O(k)

Q37. Combination Sum IV (LeetCode 377, Medium)

Source: LeetCode 377 | Pattern: 1D DP (ordered sequences)

Question: Given distinct positive integers `nums` and a `target`, return the number of **ordered** sequences that sum to `target`. Different orderings count separately.

Example: `nums=[1,2,3], target=4` gives `7`.

Approach: `dp[t] = sum(dp[t - x] for x in nums if x <= t)`, with `dp[0] = 1`. Keep the **target loop outside** so orderings are counted. Putting `nums` outside would count combinations instead (the Coin Change II pattern).

```python
```

```python
def combinationSum4(nums, target):
    dp = [0] * (target + 1)
    dp[0] = 1
    for t in range(1, target + 1):
        for x in nums:
            if x <= t:
                dp[t] += dp[t - x]
    return dp[target]
```

Complexity: Time: O(target·len(nums)) | Space: O(target)

Q38. Warehouse Loading / Bank Transactions: Can a Prefix or Subset Reach a Target?

Source: Confluent / IBM | Pattern: Subset-sum with a set of reachable sums

Question: Given weights `W` (positive = load, negative = unload), you may run the operations in **any order**. Return whether, after some prefix, the running load equals `target` exactly. (Bank version: does any subset of transactions sum to the target balance?)

Example: `[2,-1,4], target=1` is `True` (order `-1, 2`). `[-3,5], target=2` is `True`. `[7,-3,5,-2], target=4` is `True` (`7 + -3`). `[2], target=1` is `False`.

Approach:

1. Any subset can be placed first in the ordering, so the question is **subset sum** (the empty prefix gives 0).
2. Keep a set `reachable` of sums, starting at `{0}`. For each weight add `s + w` for every `s`.
3. Stop early once `target` appears.
4. If the load may never go below 0, the answer is the same when `target >= 0` (put the subset's positives first, then its negatives, the load only falls down to `target`), and `False` when `target < 0`.

python

```python
def can_reach(weights, target):
    if target == 0:
        return True                                  # empty prefix
    reachable = {0}
    for w in weights:
        reachable |= {s + w for s in reachable}
        if target in reachable:
            return True
    return False
```

Complexity: Time: O(N·S) | Space: O(S)

Watch out: Ask about value ranges: small range means this set/DP, tiny N means brute force or meet-in-the-middle, and "improve on all subsets" is exactly this DP.

Q39. Best Price with Value Meals (up to 3 unique wanted items)

Source: Confluent (LeetCode discuss) | Pattern: Bitmask DP

Question: A menu lists `[price, "item, item, ..."]`. Single items and value meals (bundles at a discount) are all offers. Given the items the user wants (at most 3 unique), return the minimum total price to get all of them. A bundle may contain items the user does not want.

Example: Offers: `[5,"pizza"] [8,"sandwich, coke"] [4,"pasta"] [2,"coke"] [6,"pasta, coke, pizza"] [8,"burger, coke, pizza"] [5,"sandwich"]`, wants `["burger","pasta"]` gives `12` (burger meal 8 + pasta 4).

Approach:

1. Give each wanted item a bit. For each offer compute `mask` = wanted items it covers. Skip offers with `mask == 0`.
2. `dp[mask]` = cheapest price to cover exactly the wanted items in `mask`. `dp[0] = 0`.
3. Walk masks upward; for each offer: `dp[mask | m] = min(dp[mask | m], dp[mask] + price)`.
4. Answer `dp[full]` (or `-1` if unreachable).

python

```
def best_price(offers, wants):
    idx = {w: i for i, w in enumerate(wants)}
    full = (1 << len(wants)) - 1
    INF = float("inf")
    dp = [INF] * (full + 1)
    dp[0] = 0
    bundles = []
    for price, items in offers:
        m = 0
        for name in items.split(","):
            name = name.strip()
            if name in idx:
                m |= 1 << idx[name]
        if m:
            bundles.append((m, price))
    for mask in range(full + 1):                    # masks only grow, so ascending
        if dp[mask] == INF:
            continue
        for m, price in bundles:
            nm = mask | m
            dp[nm] = min(dp[nm], dp[mask] + price)
    return dp[full] if dp[full] != INF else -1
```

Complexity: Time: O(2^k · offers), k ⩽ 3 | Space: O(2^k)

Watch out: Follow-up with quantities (`2 burgers, 1 fries`): use memoized recursion over the remaining-quantity tuple.

python

```python
from functools import lru_cache

def best_price_qty(menu, meals, order):
    # menu {item: price}, meals [({item: qty}, price)], order {item: qty}
    items = [k for k, q in order.items() if q > 0]
    idx = {k: i for i, k in enumerate(items)}
    opts = [(tuple(int(j == idx[k]) for j in range(len(items))), p)
            for k, p in menu.items() if k in idx]
    for bundle, p in meals:
        v = [0] * len(items)
        for k, q in bundle.items():
            if k in idx:
                v[idx[k]] = q
        if any(v):
            opts.append((tuple(v), p))

    @lru_cache(None)
    def f(need):
        if not any(need):
            return 0
        best = float("inf")
        for v, p in opts:
            nxt = tuple(max(0, n - x) for n, x in zip(need, v))
            if nxt != need:
                best = min(best, p + f(nxt))
        return best

    return f(tuple(order[k] for k in items))
```

Complexity of the quantity version: states are the product of `(qty+1)` per item, each tries every offer.

Q40. Minimum Number of Days to Eat N Oranges (LeetCode 1553, Hard)

Source: LeetCode 1553 | Pattern: Memoized recursion

Question: Each day you may eat 1 orange, or `n/2` oranges if `n` is divisible by 2, or `2n/3` oranges if `n` is divisible by 3. Return the minimum days to eat all `n` oranges.

Example: `n=10` gives `4` (10 → 9 → 3 → 1 → 0). `n=6` gives `3`.

Approach: Only two useful moves: halve, or take two thirds. Use single "eat 1" days to reach a divisible number. So `f(n) = 1 + min(n % 2 + f(n // 2), n % 3 + f(n // 3))`,

memoized.

```python
from functools import lru_cache

def minDays(n):
    @lru_cache(None)
    def f(n):
        if n <= 1:
            return n
        return 1 + min(n % 2 + f(n // 2), n % 3 + f(n // 3))
    return f(n)
```

Complexity: Time: O(log² n) | Space: O(log² n)

Q41. Word Break (LeetCode 139, Medium)

Source: LeetCode 139 | Pattern: 1D DP

Question: Given a string `s` and a dictionary `wordDict`, return whether `s` can be split into a sequence of dictionary words (words may be reused).

Example: `s="leetcode", ["leet","code"]` is True. `s="catsandog", ["cats","dog","sand","and","cat"]` is False.

Approach: `dp[i]` means the prefix `s[:i]` can be segmented. `dp[i]` is True if some `j < i` has `dp[j]` and `s[j:i]` is a word. Only check `j` within the longest word length.

```python
def wordBreak(s, wordDict):
    words = set(wordDict)
    maxlen = max(map(len, words))
    dp = [False] * (len(s) + 1)
    dp[0] = True
    for i in range(1, len(s) + 1):
        for j in range(max(0, i - maxlen), i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[-1]
```

Complexity: Time: O(n·L), L = longest word | Space: O(n)

Q42. Course Schedule (LeetCode 207, Medium)

Source: LeetCode 207 | Pattern: Graph, topological sort (Kahn)

Question: There are `numCourses` courses and prerequisite pairs `[a, b]` meaning you must take `b` before `a`. Return whether all courses can be finished, which means the graph has no cycle.

Example: `n=2, [[1,0]]` is True. `n=2, [[1,0],[0,1]]` is False.

Approach: Kahn's algorithm (BFS topological sort). If every course gets processed, there is no cycle.

```python
from collections import deque

def canFinish(numCourses, prerequisites):
    g = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(numCourses) if indeg[i] == 0)
    done = 0
    while q:
        u = q.popleft()
        done += 1
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return done == numCourses
```

Complexity: Time: O(V + E) | Space: O(V + E)

Q43. Course Schedule II (LeetCode 210, Medium)

Source: LeetCode 210 | Pattern: Graph, topological sort (Kahn)

Question: Same setup as 207, but return a valid order to take all courses, or `[]` if impossible.

Example: `n=4, [[1,0],[2,0],[3,1],[3,2]]` gives `[0,1,2,3]` or `[0,2,1,3]`.

Approach: Kahn's algorithm again, recording the processing order.

```python
from collections import deque

def findOrder(numCourses, prerequisites):
    g = [[] for _ in range(numCourses)]
    indeg = [0] * numCourses
    for a, b in prerequisites:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(numCourses) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == numCourses else []
```

Complexity: Time: O(V + E) | Space: O(V + E)

Q44. Minimum Monster Cost in a Grid (+ path)

Source: Confluent VO (reconstructed) | Pattern: Dijkstra

Question (reconstructed): The original wording is only a vague summary ("graph search and shortest path with path reconstruction"). Assumed: a grid where entering a cell costs the monster there (`-1` = wall). Move up, down, left, right from `start` to `goal`. Return the minimum total cost and the path. **Confirm the rules** in the interview.

Example: `grid=[[1,3,1],[1,5,1],[4,2,1]]`, `start=(0,0)`, `goal=(2,2)` gives cost `7` with path `(0,0),(0,1),(0,2),(1,2),(2,2)`.

Approach:

1. Cells have different costs, so plain BFS is wrong. Use Dijkstra with a min-heap of `(cost, r, c)`.
2. Skip popped entries that are worse than the recorded `dist`.
3. On every improvement store `parent[next] = current`.
4. Rebuild the path by walking parents back from the goal.

5. Only right/down moves allowed: plain DP `dp[r][c] = cost + min(top, left)` is enough.

python

```python
import heapq

def min_monster_cost(grid, start, goal):
    R, C = len(grid), len(grid[0])
    INF = float("inf")
    dist = [[INF] * C for _ in range(R)]
    parent = {}
    sr, sc = start
    dist[sr][sc] = grid[sr][sc]
    pq = [(grid[sr][sc], sr, sc)]
    while pq:
        d, r, c = heapq.heappop(pq)
        if d > dist[r][c]:
            continue
        if (r, c) == goal:
            break
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] >= 0:
                nd = d + grid[nr][nc]
                if nd < dist[nr][nc]:
                    dist[nr][nc] = nd
                    parent[(nr, nc)] = (r, c)
                    heapq.heappush(pq, (nd, nr, nc))
    gr, gc = goal
    if dist[gr][gc] == INF:
        return -1, []
    path, cur = [goal], goal
    while cur != start:
        cur = parent[cur]
        path.append(cur)
    return dist[gr][gc], path[::-1]
```

General Dijkstra template (adjacency list `graph = {node: [(neighbor, weight), ...]}`):

python

```python
import heapq

def dijkstra(graph, src):
    dist = {u: float("inf") for u in graph}
    dist[src] = 0
    prev = {u: None for u in graph}
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v], prev[v] = d + w, u
                heapq.heappush(pq, (d + w, v))
    return dist, prev
```

Complexity: Time: O(R·C·log(R·C)) | Space: O(R·C)

Watch out: Dijkstra needs non-negative costs. A state-dependent constraint (e.g. remaining health) goes into the heap state: `(cost, r, c, extra)`.

Q45. Eliminate Maximum Number of Monsters (LeetCode 1921, Medium)

Source: LeetCode 1921 | Pattern: Greedy + sort

Question: `dist[i]` is the distance of monster `i` from the city and `speed[i]` its speed per minute. You have a weapon that kills one monster, then needs one minute to recharge (it starts charged). You lose if a monster reaches the city. Return the maximum number of monsters you can kill before losing.

Example: `dist=[1,3,4], speed=[1,1,1]` gives `3`. `dist=[1,1,2,3], speed=[1,1,1,1]` gives `1`.

Approach: Arrival time is `ceil(dist / speed)`. Kill in order of arrival. The i-th shot (0-indexed) happens at minute `i`, so you lose if `arrival <= i`.

python

```python
def eliminateMaximum(dist, speed):
    times = sorted((d + s - 1) // s for d, s in zip(dist, speed))
    for i, t in enumerate(times):
        if t <= i:
            return i
    return len(times)
```

Complexity: Time: O(n log n) | Space: O(n)

Q46. Minimum Initial Strength to Defeat All Monsters (LeetCode 4008, Medium)

Source: LeetCode 4008 | Pattern: Binary search on answer + difference array

Question (reconstructed): `monsters[i]` is the strength of the i-th monster. `boosts[j] = [l, r, v]` adds `v` to the bonus of every monster index in `[l, r]`, and overlapping boosts add up. You start with some non-negative strength and fight left to right. For monster `i`, you win if `strength + bonus[i] >= monsters[i]`. After the fight, `strength = max(0, strength - monsters[i])`. The bonus is temporary. Return the minimum initial strength that defeats every monster.

> Caveat: the LeetCode page wasn't readable, so I rebuilt the rules from a third-party mirror whose text was partly garbled. I checked the rules against both examples below. Check them against the official statement before relying on this.

Example 1: `monsters=[5,10,15], boosts=[[1,1,10]]` gives `30`. **Example 2:** `monsters=[5,10,15], boosts=[[1,2,10],[1,2,5]]` gives `5`.

Approach: More starting strength never hurts, so the answer is monotonic. Binary search on the start value. Build `bonus[]` with a difference array, then simulate. Starting with `sum(monsters)` always works, so it is the upper bound.

python

```python
def minimumInitialStrength(monsters, boosts):
    n = len(monsters)
    diff = [0] * (n + 1)
    for l, r, v in boosts:
        diff[l] += v
        diff[r + 1] -= v
    bonus, cur = [0] * n, 0
    for i in range(n):
        cur += diff[i]
        bonus[i] = cur

    def can(s):
        for i in range(n):
            if s + bonus[i] < monsters[i]:
                return False
            s = max(0, s - monsters[i])
        return True

    lo, hi = 0, sum(monsters)
    while lo < hi:
        mid = (lo + hi) // 2
        if can(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Complexity: Time: O(n + b + n log S), S = sum(monsters) | Space: O(n)

Q47. Minimum Time to Eat All Grains (LeetCode 2604, Hard)

Source: LeetCode 2604 | Pattern: Binary search on answer + greedy sweep

Question: Hens and grains sit on a number line. A hen moves 1 unit per second and eats a grain instantly on arrival. All hens move at the same time. Return the minimum time to eat all grains.

Example: `hens=[3,6,7], grains=[2,4,7,9]` gives `2`. `hens=[4,6,109,111,213,215], grains=[5,110,214]` gives `1`.

Approach: Binary search on time `t`. For feasibility, sort both arrays and sweep hens left to right with a pointer `j` at the leftmost uneaten grain. If it is left of the hen at distance `d`, the

hen must reach it (`d <= t`). Its remaining reach to the right is `max(t - 2d, (t - d) // 2)`. Otherwise the hen reaches `h + t`. Advance `j` past everything covered.

python

```python
def minimumTime(hens, grains):
    hens.sort(); grains.sort()
    m = len(grains)

    def can(t):
        j = 0
        for h in hens:
            if j == m:
                return True
            if grains[j] < h:
                d = h - grains[j]
                if d > t:
                    return False
                reach = h + max(t - 2 * d, (t - d) // 2)
            else:
                reach = h + t
            while j < m and grains[j] <= reach:
                j += 1
        return j == m

    lo, hi = 0, 2 * (max(hens[-1], grains[-1]) - min(hens[0], grains[0]))
    while lo < hi:
        mid = (lo + hi) // 2
        if can(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Complexity: Time: O(n log n + m log m + (n + m) log T) | Space: O(1) extra

---

Q48. Minimum Value to Get Positive Step-by-Step Sum

Source: LeetCode 1413 | Pattern: Prefix sum minimum

Question: Find the smallest positive integer `x` such that `x` plus every running total of `nums` (left to right) stays at least `1`.

Example: `[-3,2,-3,4,2]` gives `5` (running totals 2, 4, 1, 5, 7). `[1,2]` gives `1`. `[1,-2,-3]` gives `5`.

Approach:

1. Running total after step i is `x + prefix[i]`.
2. Need `x + minPrefix >= 1`, so `x = 1 - minPrefix`, and `x >= 1`.
3. Track the minimum prefix (start it at 0) in one pass.

```python
def minStartValue(nums):
    prefix = min_prefix = 0
    for v in nums:
        prefix += v
        min_prefix = min(min_prefix, prefix)
    return max(1, 1 - min_prefix)
```

Complexity: Time: O(n) | Space: O(1)

Appendix A: Problems in your notes that I did not solve

These had no usable statement, or are not coding questions.

Item (from `online_question.py`)	What it is	What to do
IBM "Maximum Concurrent Processes" (link only)	No statement in the file. It is almost certainly "max overlapping intervals".	Use Q16 (Meeting Rooms II): sort by start, min-heap of end times, answer is the peak heap size.
"Create a data structure for CRUD on data in a time period"	Vague	Treat as Q5 (time-based key-value store) or Q9 (sliding-window cache). Clarify.
"Java concurrency with threads"	Not specified	See thread-safety parts of Q10 (SafeLogger) and Q4 (RandomQueue): lock around check-and-update.
Phone screen: "similar to Time Based Key-Value Store, maintain a sum and average"	Variant	Q5 plus a running `sum` and `count`, or Q9.
Coding round 2: "similar to Combination Sum with memoization follow-ups"	Variant	Q34 to Q37. Memoize on `(index, remaining)` for the counting version.
System design: podcast service (feed generation), TinyURL, RSS news feed	Not coding	Prepare separately (feed fan-out on write vs read, ID generation, caching).
Behavioral / values rounds	Not coding	Prepare project deep dives and trade-offs.

Topic priority from your notes: DP (1D, 2D, bitmask), graphs (BFS/DFS, topological sort, union-find), binary search, backtracking and recursion, trie, segment trees, sliding window, bit manipulation.

Appendix B: Corrections I made to the code in `online_question.py`

I rewrote these solutions rather than copying them, because of the issues below.

- **Warehouse Loading class version:** the method had no `self`, and it only followed paths where the new load was `> 0`, which wrongly rejects valid cases such as `[-3, 5]`. The permutation search over bitmasks is also unnecessary: the problem reduces to subset sum (Q38).

- **Count events in window:** the code's claimed output `[2,2,3,2]` for `[1,1,2,3], W=1` contradicts its own explanation of `(t-W, t]`. The correct output is `[2,2,1,1]` (Q15 uses two pointers and gets both samples right).
- **TokenManager demo comment:** it says only `token_A` is active at `t=112`. Token B was created at 105 with TTL 10, so it expires at 115 and is still active: the correct output is `[A, B]` (Q8).
- **SensorHealth:** the docstring says recording a ping is O(log P). `bisect.insort` is O(P) because it shifts elements; the query is O(log P) (Q11).
- **Best price (no-`lru_cache` version):** the second `def get_best_price` is mis-indented inside the earlier `if __name__ == "__main__":` block, so the file does not run as written. Q39 has a clean bitmask solution plus a quantity version.
- **Key-value store with max (DLL version):** it claims all operations are O(1), but inserting into the sorted list is O(N) in the worst case. Q6 uses a lazy-deletion heap, with a note on why strict O(1) max under arbitrary updates is not achievable.
- `sortedcontainers`: several solutions import it, which is not in the standard library and may be unavailable in an interview. The versions here use only the standard library.

Appendix C: Reconstructed or unverified problems

- **Q44 (Monster cost grid) and Q23 (Minimum health for gaming):** the sources only gave one-line summaries, so the rules are my best reading. Confirm them with the interviewer.
- **LeetCode 4008 (Q46):** the statement was rebuilt from a third-party mirror. Check it against the official text before relying on it.
- **LeetCode 2604 (Q47):** a premium problem, with the statement paraphrased from your notes.